Apache Pulsar: Middleware para Mensageria e Streaming

Estudo de Caso: Pulsar Investimentos

- João Leão Guedes Filho
- Breno Vieira Nogueira Carneiro
- Wesley Batista Luz
- Pedro Venâncio dos Santos
- Rodrigo Braz de Oliveira

Contexto e Motivação

O Apache Pulsar surgiu da necessidade de superar limitações críticas de escalabilidade e confiabilidade em sistemas de grande escala, onde soluções tradicionais falhavam em gerenciar milhões de tópicos com baixa latência.

Característica	Descrição Técnica e Histórica
Origem	Desenvolvido pelo Yahoo (2010) para suportar serviços como Yahoo Mail e Finance. Doado à Apache Software Foundation em 2016.
Objetivo	Prover uma plataforma unificada que combine mensageria de alto desempenho com streaming de eventos em tempo real.
Cenário Prático	**Pulsar Investimentos:** Distribuição instantânea de alertas financeiros e oportunidades de mercado para milhares de clientes.

O Problema e o Middleware

A comunicação direta entre serviços cria um acoplamento forte, onde falhas, indisponibilidade ou lentidão de um componente impactam diretamente todo o sistema distribuído.

> *O Middleware Orientado a Mensagens (MOM) atua como uma camada intermediária responsável pelo roteamento, persistência e entrega confiável das mensagens entre processos distribuídos.*

Comunicação Direta

- Alto acoplamento entre serviços;
- Dependência de disponibilidade simultânea;
- Baixa tolerância a falhas;
- Dificuldade de escalabilidade.

Middleware Orientado a Mensagens

- Comunicação assíncrona;
- Persistência de mensagens;
- Intermediação via broker;
- Maior flexibilidade e resiliência.

Paradigma Publish-Subscribe (Pub/Sub)

No modelo Publish-Subscribe, produtores e consumidores se comunicam de forma indireta através de tópicos, promovendo o desacoplamento fraco entre as aplicações.

> *O desacoplamento permite que produtores e consumidores operem sem conhecer a localização, identidade ou estado uns dos outros.*

Produtores (Publishers)

- Publicam eventos em tópicos;
- Não conhecem os consumidores;
- Apenas enviam mensagens ao broker.

Consumidores (Subscribers)

- Assinam tópicos de interesse;
- Recebem mensagens assincronamente;
- Processam eventos de forma independente.

O Apache Pulsar unifica mensageria e streaming em uma única plataforma distribuída.

Desacoplamento no Tempo e Espaço

No Espaço

Os componentes do sistema **não precisam conhecer a localização física**, endereços IP ou identificadores específicos uns dos outros.
A comunicação é mediada por tópicos lógicos, permitindo que a infraestrutura mude sem impactar o código da aplicação.

No Tempo

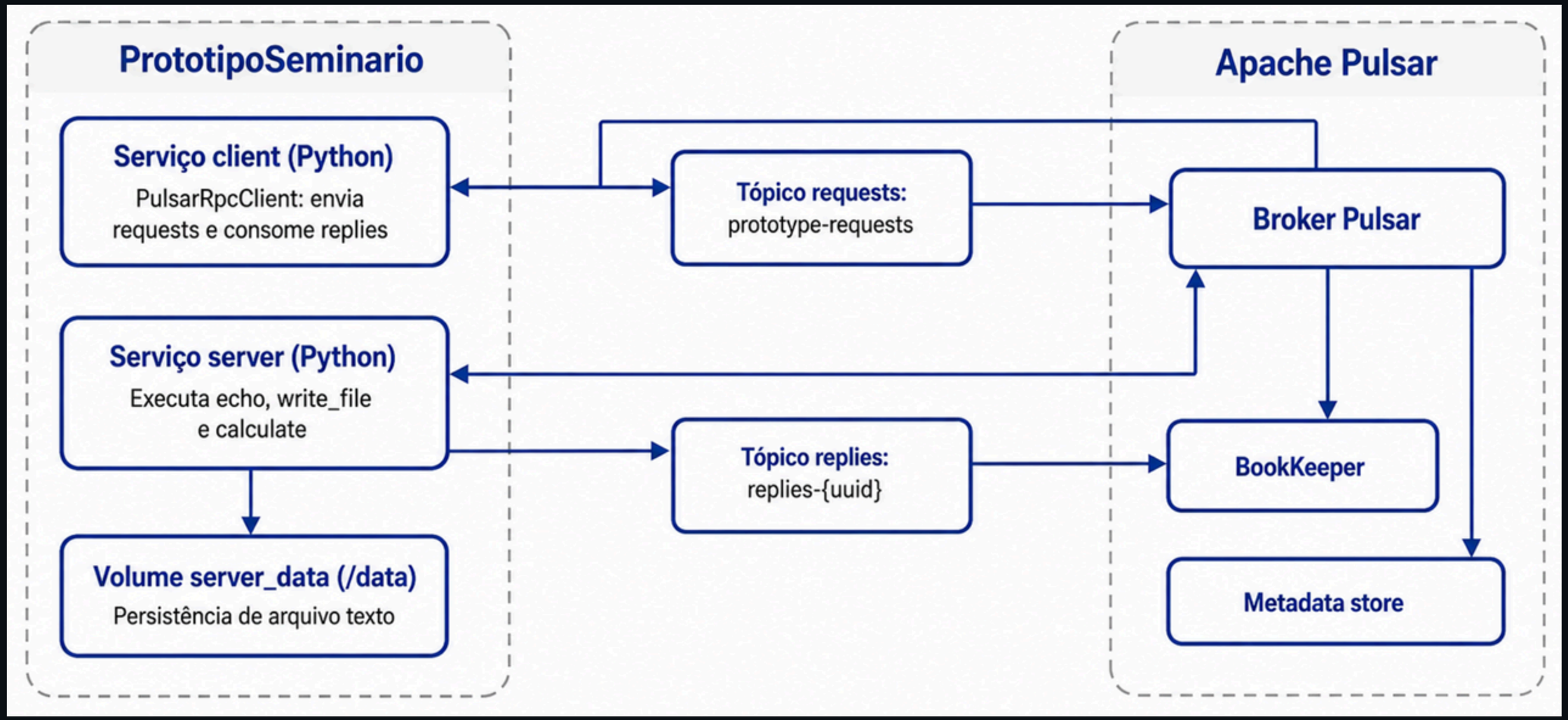
O emissor e o receptor **não precisam estar ativos simultaneamente** para que a troca de mensagens ocorra.
O Pulsar garante a persistência das mensagens até que o consumidor esteja pronto para processá-las, suportando picos de carga e falhas temporárias.

Arquitetura Multicamadas

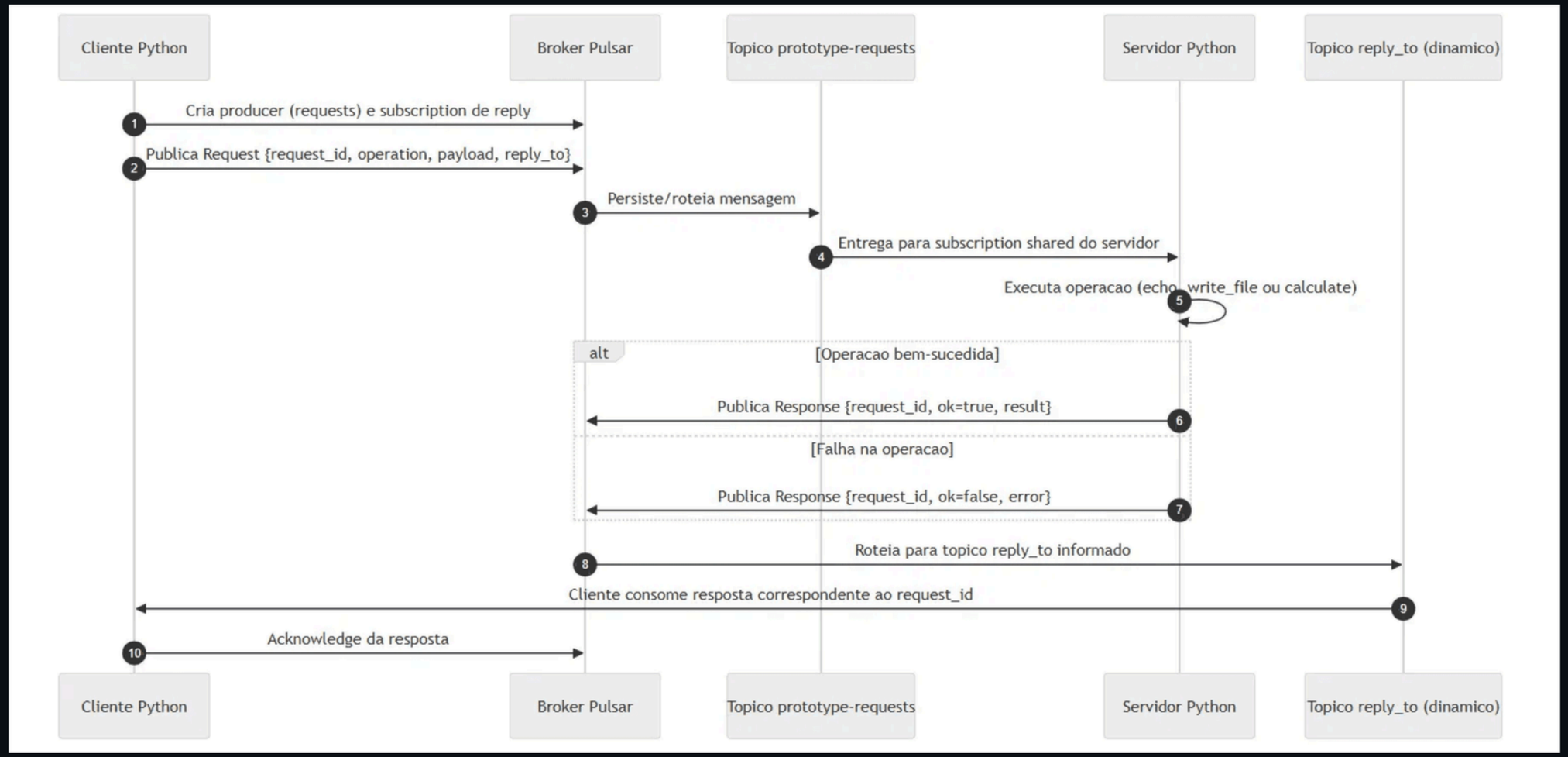
A arquitetura do Apache Pulsar separa a computação do armazenamento, permitindo escalabilidade independente.

CAMADA	TECNOLOGIA	FUNÇÃO PRINCIPAL
Serving Layer	**Brokers Stateless**	Gerencia conexões, roteamento e despacho para consumidores; escala horizontalmente por ser sem estado.
Storage Layer	**Apache BookKeeper**	Camada de armazenamento responsável por persistir as mensagens nos ledgers, replicados entre múltiplos nós chamados bookies.
Metadata Store	**Apache ZooKeeper, etcd, RocksDB ou Oxia**	Coordenação do cluster, metadados de tópicos e configurações globais para consistência do estado.

Modelagem da Solução (Visão de Componentes)



Fluxo de Comunicação (Request/Reply)



O fluxo detalha a interação assíncrona mediada pelo **Broker Pulsar**, desde a criação do produtor pelo cliente até o **Acknowledge** final da resposta.

Validação e Testes

ID	Objetivo do Teste	Status
T1	Resposta de texto (echo)	PASS
T2	Alteração de arquivo (write_file)	PASS
T3	Cálculo de função (calculate)	PASS
T4	Tratamento de erro (função inválida)	PASS
T5	Persistência no volume do servidor	PASS

Os testes confirmam o **desacoplamento total** entre cliente e servidor, com o Broker Pulsar gerenciando o roteamento e a persistência de todas as mensagens.

Evidências Técnicas

```
# Log do Servidor
Recebido echo id=1e3c60...
reply_to=replies-96e357...

# Log do Broker
Created topic persistent://...
replies-96e357...

# Retorno do Cliente
{ "ok": true, "result": 42 }
```

Criação dinâmica de tópicos e correlação via *request_id* validadas nos logs.

Análise Crítica

Vantagens

- **Multi-tenancy nativo:** Isolamento lógico para diferentes equipes ou aplicações.
- **Escalabilidade horizontal:** Separação de camadas facilita o crescimento sob demanda.
- **Persistência robusta:** Garantia de entrega e durabilidade via BookKeeper.

Limitações

- **Complexidade operacional:** Exige gestão de múltiplos componentes (ZooKeeper, Bookies).
- **Curva de aprendizado:** Configuração inicial mais densa que brokers tradicionais.
- **Overhead:** Pode ser excessivo para sistemas pequenos e síncronos.

Cenários de Uso

- **Recomendado:** Integração de microsserviços, IoT e sistemas financeiros de alto volume.
- **Evitar:** Aplicações monolíticas simples ou sistemas com baixa concorrência.

Papéis e Responsabilidades

Este projeto foi desenvolvido de forma colaborativa, com atribuições específicas para otimizar a contribuição individual e garantir a abrangência do trabalho.

Integrante	Papel Principal	Responsabilidades Específicas
João Leão	**Abertura, Organização e Fechamento**	Introdução, coordenação geral, estruturação do roteiro e conclusão do seminário.
Rodrigo Braz	**Problema, Paradigma e Modelagem**	Análise do problema, explicação do paradigma Pub/Sub e modelagem da solução.
Breno Vieira	**Arquitetura e Fluxo**	Detalhamento da arquitetura multicamadas e descrição do fluxo de comunicação.
Pedro Venâncio	**Implementação**	Desenvolvimento do protótipo (cliente/servidor), codificação das operações.
Wesley Batista	**Testes, Validação e Análise Crítica**	Execução e validação dos testes, análise crítica e cenários de uso.

Princípios Colaborativos

- Compreensão Abrangente: Entendimento completo da tecnologia e conceitos.
- Participação Ativa: Contribuição para todos os materiais e feedback mútuo.

Conclusão e Referências

Conclusão

O protótipo validou a eficácia do Apache Pulsar como middleware para sistemas distribuídos, garantindo o desacoplamento total e a persistência de mensagens. A arquitetura multicamadas provou ser uma solução robusta para cenários de alta escalabilidade, como o de investimentos financeiros.

Repositório do Projeto:

github.com/pdrVenancio/PrototipoSeminario

Referências

- **Apache Pulsar (2026).** Python client library documentation. Disponível em: pulsar.apache.org
- **Apache Pulsar (2026).** Run pulsar in standalone mode with docker. Disponível em: pulsar.apache.org/docs/en/standalone-docker/
- **PyPI (2026).** pulsar-client 3.5.0. Disponível em: pypi.org/project/pulsar-client/
- **UNIFEI (2026).** Material da disciplina XRSC09 - Sistemas Distribuídos. Prof. Rafael Frinhani.

Obrigado!